2. Exigences du projet

2.1 Exigences fonctionnelles (EF)

Code	Exigence fonctionnelle	Acteur principal
EF01	Se connecter et se déconnecter avec un compte autorisé.	Tous les utilisateurs
EF02	Consulter un tableau de bord avec les réparations en cours, véhicules reçus et factures impayées.	Administrateur, réceptionniste
EF03	Créer, consulter, modifier et rechercher les fiches clients.	Réceptionniste, administrateur
EF04	Associer un ou plusieurs véhicules à un client et gérer immatriculation, marque, modèle et kilométrage.	Réceptionniste, administrateur
EF05	Créer un ordre de réparation avec date d'entrée, panne décrite, date de sortie prévue et statut.	Réceptionniste
EF06	Affecter un mécanicien à un ordre et permettre le suivi des étapes de réparation.	Réceptionniste, mécanicien
EF07	Ajouter des lignes de réparation (main-d'œuvre et/ou pièces), quantité et prix unitaire; calculer le sous-total.	Mécanicien, réceptionniste
EF08	Gérer les pièces de rechange : ajouter, modifier, consulter le stock et signaler un stock faible.	Administrateur, réceptionniste
EF09	Décrémenter le stock lorsqu'une pièce est utilisée et empêcher une sortie supérieure au stock disponible.	Système, réceptionniste
EF10	Générer une facture à partir d'un ordre terminé, calculer le total et enregistrer le statut de paiement.	Réceptionniste, administrateur
EF11	Consulter l'historique des réparations d'un véhicule ou d'un client.	Réceptionniste, administrateur
EF12	Consulter des statistiques simples : nombre d'ordres, chiffre d'affaires enregistré et factures impayées.	Administrateur
EF13	Créer, modifier, désactiver et attribuer un rôle aux comptes utilisateurs.	Administrateur
EF14	Valider les formulaires et afficher un message clair en cas d'erreur ou de succès.	Système

2.2 Exigences non fonctionnelles (ENF)

Code	Exigence / critère attendu
ENF01 — Ergonomie	Interface claire, libellés en français, navigation cohérente et formulaires faciles à comprendre.
ENF02 — Sécurité	Mot de passe stocké sous forme de hash; contrôle des rôles; aucune fonction interdite à un rôle non autorisé.
ENF03 — Intégrité	Clés primaires/étrangères, champs obligatoires et règles de cohérence pour éviter les données orphelines.
ENF04 — Fiabilité	Les opérations liées (facture, lignes et stock) doivent être transactionnelles lorsque nécessaire.
ENF05 — Performance	Les listes doivent rester utilisables avec plusieurs milliers d'enregistrements sur un ordinateur de bureau standard.
ENF06 — Maintenabilité	Séparer interface, contrôleurs, services métier et accès aux données (DAO/Repository).
ENF07 — Compatibilité	Application desktop JavaFX compatible avec l'environnement Java retenu par le groupe.
ENF08 — Sauvegarde	Prévoir une procédure de sauvegarde/restauration de la base PostgreSQL.
ENF09 — Traçabilité	Afficher les dates pertinentes et conserver les statuts de chaque ordre et facture.
ENF10 — Confidentialité	Ne collecter que les données nécessaires au fonctionnement du garage.

3. Cahier des charges

3.1 Contexte et objectif

GarageManager est une application desktop destinée à centraliser les informations d'un garage mécanique. Elle vise à réduire les fiches papier, faciliter le suivi des véhicules et réparations, contrôler les pièces de rechange et établir les factures.

3.2 Périmètre de la première version

Inclus : authentification, clients, véhicules, ordres de réparation, mécaniciens, lignes de réparation, stock de pièces, factures, tableau de bord et statistiques simples.

Hors périmètre pour la première version : application mobile, accès web à distance, paiement bancaire en ligne, envoi automatique de SMS/e-mails, comptabilité complète et gestion multi-agences.

3.3 Utilisateurs et responsabilités

Acteur	Responsabilités proposées
Administrateur	Gérer les comptes et rôles; accéder aux données de gestion; consulter le tableau de bord et les statistiques.
Réceptionniste	Créer les fiches clients/véhicules, ouvrir les ordres, suivre les réparations, gérer les factures et les paiements selon les droits accordés.
Mécanicien	Consulter les ordres qui lui sont affectés, ajouter les informations de réparation et mettre à jour le statut selon les règles du garage.

3.4 Contraintes techniques

• Interface desktop : JavaFX avec FXML et CSS.

• Architecture : MVC et séparation en couches; contrôleurs sans requêtes SQL directes.

• Persistance : PostgreSQL; accès via JDBC dans des classes DAO/Repository.

• Environnement, versions Java/JavaFX et outil de construction (Maven ou Gradle) à fixer par le groupe.

• GitHub pour le versionnement; ne jamais publier de mot de passe réel ou de fichier de configuration contenant des secrets.

3.5 Règles de gestion

• Un client peut posséder plusieurs véhicules; chaque véhicule est rattaché à un client.

• Un véhicule peut avoir plusieurs ordres de réparation; chaque ordre concerne un seul véhicule.

• Un ordre possède une ou plusieurs lignes de réparation; chaque ligne doit avoir une quantité et un prix unitaire valides.

• Un ordre suit les statuts EN_ATTENTE, EN_COURS, TERMINEE et LIVREE; les transitions exactes sont à valider avec le groupe.

• Une facture est liée à un ordre. Pour la première version, décider si un ordre peut générer une seule facture ou plusieurs factures.

• Le total d'une ligne = quantité × prix unitaire; le total de facture = somme des sous-totaux, avec taxes/remises seulement si elles sont prévues.

• Le stock ne peut pas devenir négatif; une pièce utilisée doit être déduite du stock de façon cohérente.

• Un compte désactivé ne peut pas se connecter; chaque écran et action doivent respecter le rôle.

4. Cas d’utilisation détaillés

Le diagramme de cas d’utilisation du PDF est une bonne ébauche, mais il faut rendre visibles les associations entre chaque acteur et les fonctionnalités auxquelles il a droit. Scénarios prioritaires à documenter :

UC01 — Se connecter

• Acteur : administrateur, réceptionniste ou mécanicien.

• Précondition : un compte actif existe.

• Scénario nominal : l’utilisateur saisit identifiant et mot de passe; le système vérifie les informations et ouvre l’espace correspondant au rôle.

• Alternative : informations incorrectes ou compte désactivé → message d’erreur sans révéler le mot de passe.

• Postcondition : session ouverte; les menus affichés dépendent du rôle.

UC02 — Créer un ordre de réparation

• Acteur : réceptionniste.

• Précondition : le client et le véhicule existent; sinon, ils doivent être créés d’abord.

• Scénario nominal : sélectionner le véhicule, saisir la panne, la date d’entrée et la date de sortie prévue, puis enregistrer l’ordre en statut EN_ATTENTE.

• Alternative : champ obligatoire manquant ou date invalide → signaler les champs à corriger.

• Postcondition : ordre enregistré et visible dans la liste des réparations.

UC03 — Ajouter une pièce à une réparation

• Acteur : mécanicien ou réceptionniste autorisé.

• Précondition : ordre ouvert et pièce disponible en stock.

• Scénario nominal : choisir la pièce, saisir la quantité, vérifier le stock et le prix, puis confirmer.

• Alternative : quantité supérieure au stock → refuser l’opération et afficher le stock disponible.

• Postcondition : ligne de réparation créée et stock mis à jour selon la règle validée.

UC04 — Générer une facture

• Acteur : réceptionniste ou administrateur autorisé.

• Précondition : ordre terminé et lignes de réparation renseignées.

• Scénario nominal : calculer les sous-totaux, calculer le total, générer la facture et la marquer comme non payée.

• Alternative : facture déjà générée ou ordre incomplet → empêcher le doublon et expliquer le problème.

• Postcondition : facture consultable; paiement enregistré séparément lorsqu’il est reçu.

UC05 — Mettre à jour le statut d’une réparation

• Acteur : mécanicien pour les ordres qui lui sont affectés; autres rôles selon les droits.

• Scénario nominal : ouvrir l’ordre, sélectionner un statut autorisé et enregistrer.

• Alternative : transition non autorisée ou ordre inexistant → refuser et afficher une explication.

• Postcondition : le nouveau statut apparaît dans le suivi et le tableau de bord.

À ajouter au diagramme UML : relier explicitement les trois acteurs à leurs cas d’utilisation; représenter éventuellement « Générer facture » comme dépendant de « Calculer total », et « Ajouter pièce » comme dépendant de « Vérifier stock », si votre enseignant accepte les relations include.

6. Maquettes UI — wireframes indicatifs

Le PDF initial nomme les écrans, mais ne contient pas de véritables maquettes d'interface. Les wireframes ci-dessous indiquent les éléments à reproduire dans Figma ou un outil équivalent. Ils sont des plans fonctionnels, pas des captures d'un prototype déjà construit.

ÉCRAN 1 — Connexion
GarageManager | Connexion à votre espace
Nom d'utilisateur : [_____]
Mot de passe : [_____] [Afficher/masquer]
[Se connecter]
Message d'erreur affiché ici si les identifiants sont invalides

ÉCRAN 2 — Tableau de bord
Menu : Tableau de bord | Clients | Véhicules | Réparations | Stock | Factures | Utilisateurs
Cartes : Ordres en cours [N] Véhicules reçus [N] Factures impayées [N]
Liste : réparations récentes (N° ordre, véhicule, client, statut, date prévue)
Alertes : pièces sous le seuil de stock

ÉCRAN 3 — Ordres de réparation
Filtres : N° ordre [___] Client [___] Statut [Tous ▼] [Rechercher]
Tableau : N° | Client | Véhicule | Date d'entrée | Statut | Mécanicien | Actions
[+ Nouvel ordre] [Consulter] [Modifier statut]
Formulaire : véhicule, panne, dates, mécanicien, lignes de réparation

ÉCRAN 4 — Stock et factures
Stock : rechercher une pièce | nom | prix | quantité | seuil d'alerte | [Ajouter] [Modifier]
Factures : N° facture | ordre | date | total | statut paiement | [Détails]
Détail facture : lignes, quantités, prix unitaires, sous-totaux, total
[Enregistrer paiement] [Imprimer/Exporter si cette fonction est retenue]

Pour Figma : créez une page par écran, gardez les mêmes couleurs, boutons et menus, ajoutez les états d'erreur/confirmation, puis partagez le lien en lecture. Les captures/exportations des maquettes doivent être ajoutées au dossier final.

7. Démarche, outils et plan de tests

7.1 Démarche proposée

Étape	Travail	Résultat attendu
1. Analyse	Identifier les acteurs, besoins, contraintes et règles métier.	Liste EF/ENF + cahier des charges
2. Modélisation	Revoir les classes, multiplicités et cas d'utilisation.	Diagrammes UML validés
3. Prototypage	Dessiner les écrans et parcours principaux.	Lien Figma + captures
4. Architecture	Définir MVC, services, DAO et schéma PostgreSQL.	Schéma technique et base initiale
5. Développement	Implémenter par fonctionnalité et versionner chaque étape.	Code source organisé
6. Tests	Tester les parcours normaux et les erreurs courantes.	Tableau de tests et corrections
7. Livraison	Rédiger la synthèse et préparer le dépôt GitHub.	Dossier final + liens

7.2 Outils envisagés

• Java + JavaFX/FXML/CSS : interface desktop.

• PostgreSQL + JDBC : stockage et requêtes SQL.

• IntelliJ IDEA ou Eclipse : environnement Java (choisir un seul outil principal).

• Scene Builder : création visuelle des vues FXML, si utilisé.

• Figma : maquettes UI et partage du prototype.

• Git + GitHub : historique des modifications et partage du code.

• PlantUML, diagrams.net ou outil équivalent : création/ajustement des diagrammes UML.

7.3 Plan de tests minimal

ID	Test	Résultat attendu
T01	Connexion avec identifiants corrects	Accès à l'espace correspondant au rôle.
T02	Connexion avec mot de passe incorrect	Connexion refusée et message compréhensible.
T03	Créer un véhicule sans client	Validation refusée ou sélection d'un client obligatoire.
T04	Créer un ordre avec champs obligatoires manquants	Enregistrement refusé avec indication des champs.
T05	Utiliser plus de pièces que le stock disponible	Opération refusée; stock inchangé.
T06	Calculer une ligne de 3 unités à 20 MAD	Sous-total = 60 MAD.
T07	Générer deux fois la facture du même ordre	Le doublon est empêché si la règle est une facture par ordre.
T08	Un mécanicien tente d'ouvrir la gestion des utilisateurs	Accès refusé si ce droit n'est pas accordé.